

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). $\{1, 2, 3\}$ is an example of a set, but $\{1, 2, 2\}$ is not a set. Today you will learn how to use sets in java by solving this problem.

You are given n pairs of strings. Two pairs (a, b) and (c, d) are identical if $a = c$ and $b = d$. That also implies (a, b) is not same as (b, a) . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer T denoting number of pairs. Each of the next T lines will contain two strings separated by a single space.

Constraints:

- $1 \leq T \leq 100000$
- Length of each string is atmost 5 and will consist lower case letters only.

Output Format

Print T lines. In the i_{th} line, print number of unique pairs you have after taking i^{th} pair as input.

```

19
20 //Write your code here
21 Set<List<String>> set = new HashSet<>();
22
23     for (int i = 0; i < t; i++) {
24         List<String> pair = Arrays.asList(pair_left[i], pair_right
25         [i]);
26         set.add(pair);
27         System.out.println(set.size());
28     }
29     s.close();
30
31
32

```

Line: 30 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.



In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as `LinkedList` or `ArrayDeque` classes. For example, deque can be declared as:

```
Deque deque = new LinkedList<>();
or
Deque deque = new ArrayDeque<>();
```

You can find more details about Deque [here](#).

In this problem, you are given N integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size M .

Note: Time limit is 3 second for this problem.

Input Format

The first line of input contains two integers N and M : representing the total number of integers and the size of the subarray, respectively. The next line

```
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
if (deque.size() > m) {
    int removed = deque.removeFirst();

    map.put(removed, map.get(removed) - 1);

    if (map.get(removed) == 0) {
        map.remove(removed);
    }
}

if (deque.size() == m) {
    maxUnique = Math.max(maxUnique, map.size());

    // Maximum possible unique elements is m
    if (maxUnique == m) {
        break;
    }
}
```

Line: 42 Col: 25

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Problem List

Submit

Premium

Description

Editorial

Solutions

Submissions

26. Remove Duplicates from Sorted Array

Easy

Topics

Companies

Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [1,1,2]
Output: 2, nums = [1,2,2]
```

19.5K 1.1K 630 Online

Code

Java

Auto

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         // If the array has only one element
4         if (nums.length == 1) {
5             return 1;
6         }
7
8         int k = 1; // Points to the next position for a unique element
9
10        for (int i = 1; i < nums.length; i++) {
11            if (nums[i] != nums[i - 1]) {
12                nums[k] = nums[i];
13                k++;
14            }
15        }
16
17        return k;
18    }
19 }
```

Saved Ln 19, Col 2

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums = [1,1,2]

Output

1 2

Problem List

<

>

Submit

0

Premium

Description

Editorial

Solutions

Submissions

27. Remove Element

Solved

Easy

Topics

Companies

Hint

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Code

Java

Auto

```
1 class Solution {
2     public int removeElement(int[] nums, int val) {
3         int k = 0; // Pointer for placing valid elements
4
5         for (int i = 0; i < nums.length; i++) {
6             if (nums[i] != val) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11
12        return k;
13    }
14 }
```

Saved

Ln 1, Col 1

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =

[3,2,2,3]

val =

3

5.3K

980

167 Online

Problem List

53. Maximum Subarray

Medium

Topics

Companies

Description

Editorial

Solutions

Submissions

Solved

Given an integer array `nums`, find the **subarray** with the largest sum, and return *its* sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

38.1K

571

275 Online

Code

Java

Auto

```
1 class Solution {
2     public int maxSubArray(int[] nums) {
3         int currentSum = nums[0];
4         int maxSum = nums[0];
5
6         for (int i = 1; i < nums.length; i++) {
7             currentSum = Math.max(nums[i], currentSum + nums[i]);
8             maxSum = Math.max(maxSum, currentSum);
9         }
10
11         return maxSum;
12     }
13 }
14
15
16
17
```

Saved

Ln 6, Col 25

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =

[-2,1,-3,4,-1,2,1,-5,4]

Output

Description Editorial Solutions Submissions

1732. Find the Highest Altitude

Easy Topics Companies Hint

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length `n` where `gain[i]` is the **net gain in altitude** between points `i` and `i + 1` for all $(0 \leq i < n)$. Return the **highest altitude** of a point.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: 0

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$



 3.5K 426

21 Online

Code

Java

```
1 class Solution {
2     public int largestAltitude(int[] gain) {
3         int currentAltitude = 0;
4         int maxAltitude = 0;
5
6         for(int g : gain){
7             currentAltitude += g;
8             maxAltitude= Math.max(maxAltitude,currentAltitude);
9         }
10        return maxAltitude;
11    }
12 }
13
14
15
16
17
```

Saved

Ln 8, Col 27

 Testcase Test Result

Accepted Runtime: 0 ms

 Case 1 Case 2

Input

gain =
[-5,1,5,0,-7]

Output

Problem List

<

>

🔍

🏠

🎮

🚀 Submit

📄

🌟

🔧

⚙️

🔥 0

🕒

👤

🔄

Premium

Description

Editorial

Solutions

Submissions

🔖

<

</> Code

Java

🔒 Auto

☰

🔖

🔍

🔄

🗑️

1 import java.util.*;

2

3 class Solution {

4 public List<List<String>> groupAnagrams(String[] strs) {

5

6 // HashMap to store sorted string as key

7 // and list of anagrams as value

8 Map<String, List<String>> map = new HashMap<>();

9

10 for (String str : strs) {

11

12 // Convert string to character array

13 char[] chars = str.toCharArray();

14

15 // Sort the character array

16 Arrays.sort(chars);

17

18 // Convert sorted array back to string

19 String key = new String(chars);

Saved

Ln 31, Col 2

📄 Testcase

📄 Test Result

Accepted

Runtime: 0 ms

✅ Case 1

✅ Case 2

✅ Case 3

Input

strs =

["eat","tea","tan","ate","nat","bat"]

Output

[[{"eat","tea","tan","ate","nat","bat"}]]

👍 22.3K

💬 422

🌟

📄

🕒

🟢 312 Online

49. Group Anagrams

Medium Topics Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Problem List

Submit

0

0

0

0

0

Premium

Description

Editorial

Solutions

Submissions

347. Top K Frequent Elements

Medium

Topics

Companies

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in **any order**.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`
- `k` is in the range `[1, the number of unique elements in the array]`

19.8K494310 Online

Code

Java

Auto

```
33
34
35         result[index++] = num;
36         if (index == k) {
37             break;
38         }
39     }
40 }
41
42     return result;
43 }
44
45
46
47
48
```

SavedIn 44, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1Case 2Case 3

Input

nums =

[1,1,1,2,2,3]

k =

2

Easy Topics Companies Hint

Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is $[2, 3, 5, 4, 1, 7]$.

Input: nums = [1,2,3,4,4,3,2,1], n = 4

Output: [1,4,2,3,3,2,4,1]

Input: nums = [1,1,2,2], n = 2

Output: [1,2,1,2]

- `1 <= n <= 500`
- `nums.length == 2n`
- `1 <= nums[i] <= 10^3`

Solved 

Java Auto

```

1 class Solution {
2     public int[] shuffle(int[] nums, int n) {
3         int[] result = new int[2 * n];
4         int index = 0;
5
6         for (int i = 0; i < n; i++) {
7             result[index++] = nums[i];           // x element
8             result[index++] = nums[i + n];       // y element
9         }
10
11         return result;
12     }
13 }

```

Saved

Ln 1, Col 1

Testcase > Test Result

Accepted Runtime: 0 ms

✔ Case 1 ✔ Case 2 ✔ Case 3

Input

```
nums =
[2,5,1,3,4,7]
```

$n =$
3

Output

[2,3,5,4,1,7]

77 Online

6.5K 164